

Tugas Perancangan RTL: Simulasi Rangkaian Kombinasional

September 23, 2026

Tujuan Pembelajaran

1. Menguasai penulisan kode RTL (Verilog/SystemVerilog) untuk logika kombinasional.
2. Memahami perbedaan gaya pemodelan: *Behavioral*, *Dataflow*, dan *Structural*.
3. Membuat *Testbench* yang robust untuk memverifikasi kebenaran fungsional.
4. Menganalisis hasil simulasi melalui waveform untuk mendeteksi bug logika.

Tools yang Disarankan

- Simulator: Icarus Verilog (`iverilog`), Verilator, atau ModelSim/QuartaSim.
 - Waveform Viewer: GTKWave atau DVE.
-

1 Tingkat 1: Dasar (Fundamental Logic)

Fokus: Operator dasar, kondisi sederhana, dan pengenalan testbench.

1.1 Tugas 1.1: Pembandingan Magnitudo 4-bit

Buatlah modul yang membandingkan dua input 4-bit (A dan B).

- **Spesifikasi Port:**
 - Input: input [3:0] A , B
 - Output: output reg gt , lt , eq
- **Tantangan Desain:**
 1. Implementasikan menggunakan blok `always_comb` atau `assign`.
 2. Pastikan tidak ada *latch* yang terbentuk (cover semua kondisi).
- **Tugas Simulasi:** Buat testbench yang menguji kasus: $A > B$, $A < B$, $A = B$, serta nilai ekstrem (0 dan 15).

1.2 Tugas 1.2: Multiplexer 8:1 dengan Enable

- **Spesifikasi:** 8 input data (`d[7:0]`), 3 bit selektor (`sel[2:0]`), 1 enable aktif rendah (`en_n`).
- **Tantangan Desain:** Gunakan statement `case`. Jika `en_n` tinggi, output harus 0 terlepas dari nilai selector.
- **Tugas Simulasi:** Uji perubahan output saat `sel` berubah dalam keadaan enable aktif, dan uji responsivitas saat enable dimatikan.

2 Tingkat 2: Menengah (Arithmetic & Encoding)

Fokus: Logika aritmatika manual, prioritas, dan parameterisasi.

2.1 Tugas 2.1: Carry Look-Ahead Adder (CLA) 4-bit

Implementasikan adder cepat tanpa menggunakan operator `+` bawaan untuk penjumlahan utama.

- **Konsep Kunci:**

$$G_i = A_i B_i, \quad P_i = A_i \oplus B_i, \quad C_{i+1} = G_i + P_i C_i \quad (1)$$

- **Tantangan Desain:** Tuliskan persamaan carry C_1 sampai C_4 secara eksplisit dalam kode RTL.
- **Tugas Simulasi:**
 - Verifikasi hasil penjumlahan terhadap kasus overflow.
 - Bandingkan waveform output sum dengan perhitungan manual.

2.2 Tugas 2.2: Priority Encoder Parameterizable

- **Spesifikasi:** Modul dengan parameter `WIDTH` (default 8). Output biner sesuai indeks bit tertinggi yang aktif.
- **Tantangan Desain:** Gunakan `casez` atau loop `for` dalam blok `always`. Tambahkan output `valid`.
- **Tugas Simulasi:**
 - Uji dengan pola input *one-hot* (hanya 1 bit aktif).
 - Uji dengan pola input multi-bit aktif (pastikan prioritas tertinggi yang menang).
 - Uji kondisi semua input nol (output valid harus 0).

3 Tingkat 3: Lanjutan (System Integration)

Fokus: Hierarki modul dan kompleksitas kontrol.

3.1 Tugas 3.1: ALU 8-bit dengan Flag Status

Rancang ALU kombinasional dengan operasi berikut:

Opcode	Operasi	Keterangan
3'b000	ADD	$A + B$
3'b001	SUB	$A - B$
3'b010	AND	$A \& B$
3'b011	OR	$A B$
3'b100	XOR	$A \wedge B$
3'b101	SLT	Set if $A < B$ (Signed)

Table 1: Tabel Operasi ALU

- **Output Flag:** Zero (Z), Negative (N), Overflow (V), Carry/Borrow (C).
- **Tantangan Desain:**
 1. Pisahkan logika aritmatika dan logika flag jika memungkinkan untuk keterbacaan.
 2. Perhatikan penanganan bilangan *signed* untuk operasi SLT dan Overflow.
- **Tugas Simulasi Komprehensif:**
 - Buat stimulus acak (*randomized testing*) minimal 100 siklus.
 - Verifikasi bahwa flag Z aktif hanya ketika hasil operasi adalah 0.
 - Verifikasi flag V (Overflow) pada penjumlahan/pengurangan bilangan signed.

4 Panduan Penulisan Testbench

Untuk setiap tugas, testbench wajib memenuhi kriteria berikut:

1. **Inisialisasi:** Berikan nilai awal pada semua input sebelum simulasi dimulai.
2. **Stimulus Terstruktur:** Jangan hanya memberikan satu nilai. Ubah-ubah input secara sistematis.
3. **Self-Checking (Opsional tapi Dianjurkan):** Gunakan `$display` atau `assert` untuk mencetak "PASS" atau "FAIL" secara otomatis jika hasil simulasi tidak sesuai harapan.
4. **Dump Waveform:** Pastikan terdapat perintah `$dumpfile` dan `$dumpvars` (untuk Icarus/Verilator) agar bisa dilihat.

Format Pengumpulan

Mahasiswa diminta mengumpulkan:

1. File Source Code RTL (.v / .sv)
2. File Testbench (.v / .sv)
3. Screenshot Waveform yang menunjukkan kasus-kasus kritis (misal: transisi carry, prioritas encoder, dll).
4. Penjelasan singkat mengenai bagaimana testbench Anda memastikan desain bekerja dengan benar.

*Peringatan: Pastikan tidak ada **Combinational Loop** dalam desain Anda. Simulasi akan hang atau menghasilkan nilai 'x' jika terjadi loop umpan balik tanpa register.*